> 🛠️ **2026.10 수정본 (v11)** — 원본: `ch04_08_01_Pipeline_Code.ipynb`
>
> - 원본은 저장소에 없는 `../dataset/Space_Titanic/` 폴더를 읽어서 실행되지 않았어요. 내 PC에 `data/Space_Titanic/` 이 있으면 그것을, 없으면 (코랩 등) 공개 저장소 LDJWJ/dataAnalysis 의 같은 파일을 인터넷 주소로 읽도록 바꿨어요.
> - 학습한 파이프라인(`joblib`)은 `data/Space_Titanic/` 폴더에 저장해요.
>
> 📚 **데이터 출처**: Kaggle Spaceship Titanic 대회 데이터 (대회 데이터 페이지 표기 라이선스: CC BY 4.0, 2026-10-16 확인). 이용 조건은 대회 규칙(Rules)을 따릅니다.

머신러닝 파이프라인

학습 목표

- 머신러닝 파이프라인(pipeline)과 학습한 모델을 저장과 활용하는 방법을 알아본다.

목차

01 데이터 불러오기
02 파이프라인 생성 및 학습
03 파이프라인 함수로 만들기
04 학습내용 저장 및 불러오기

01 데이터 불러오기

목차로 이동하기

In [1]:
```
import pandas as pd
from sklearn.model_selection import train_test_split

# 2026.10 수정: 데이터 위치를 한곳에서 정합니다.
#  - 내 PC에 data/Space_Titanic 폴더(또는 예전 경로 ../dataset/Space_Titanic)가 있
#  - 없으면(코랩 등) 공개 저장소 LDJWJ/dataAnalysis 에 올려 둔 같은 파일을 인터넷 주소로 읽
# 데이터 출처: Kaggle "Spaceship Titanic" 대회 (https://www.kaggle.com/competiti
import os
RAW_URL = "https://raw.githubusercontent.com/LDJWJ/dataAnalysis/main/data/Sp
DATA_DIR = next((d for d in ["data/Space_Titanic", "../dataset/Space_Titanic
                 if os.path.exists(os.path.join(d, "train.csv"))), RAW_URL)
SAVE_DIR = "data/Space_Titanic"          # 학습한 파이프라인을 저장할 폴더
os.makedirs(SAVE_DIR, exist_ok=True)
print("데이터 위치:", DATA_DIR)

train = pd.read_csv(f"{DATA_DIR}/train.csv")
```

```
test = pd.read_csv(f"{DATA_DIR}/test.csv")
sub = pd.read_csv(f"{DATA_DIR}/sample_submission.csv")

train.shape, test.shape, sub.shape
```

데이터 위치: https://raw.githubusercontent.com/LDJWJ/dataAnalysis/main/data/Space_Titanic

Out[1]: ((8693, 14), (4277, 13), (4277, 2))

In [2]:
```
train.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 8693 entries, 0 to 8692
Data columns (total 14 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   PassengerId   8693 non-null   str
 1   HomePlanet    8492 non-null   str
 2   CryoSleep     8476 non-null   object
 3   Cabin         8494 non-null   str
 4   Destination   8511 non-null   str
 5   Age           8514 non-null   float64
 6   VIP           8490 non-null   object
 7   RoomService   8512 non-null   float64
 8   FoodCourt     8510 non-null   float64
 9   ShoppingMall  8485 non-null   float64
 10  Spa           8510 non-null   float64
 11  VRDeck        8505 non-null   float64
 12  Name          8493 non-null   str
 13  Transported   8693 non-null   bool
dtypes: bool(1), float64(6), object(2), str(5)
memory usage: 891.5+ KB
```

In [3]:
```
train.head()
```

Out[3]:

	PassengerId	HomePlanet	CryoSleep	Cabin	Destination	Age	VIP	RoomService
0	0001_01	Europa	False	B/0/P	TRAPPIST-1e	39.0	False	0.0
1	0002_01	Earth	False	F/0/S	TRAPPIST-1e	24.0	False	109.0
2	0003_01	Europa	False	A/0/S	TRAPPIST-1e	58.0	True	43.0
3	0003_02	Europa	False	A/0/S	TRAPPIST-1e	33.0	False	0.0
4	0004_01	Earth	False	F/1/S	TRAPPIST-1e	16.0	False	303.0

In [4]:
```
train.columns
```

Out[4]:
```
Index(['PassengerId', 'HomePlanet', 'CryoSleep', 'Cabin', 'Destination', 'A
ge',
       'VIP', 'RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck',
       'Name', 'Transported'],
      dtype='str')
```

In [5]:
```
sel = [ 'RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck' ]
X = train[sel]
y = train['Transported']

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
```

파이프라인에 사용될 임퓨터, 스케일러, 분류 모델 정의

In [6]:
```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler
from sklearn.tree import DecisionTreeClassifier as DTC
```

In [7]:
```
imputer = SimpleImputer(strategy = "mean")
scaler = MinMaxScaler()
model = DTC()
```

02 파이프라인 생성 및 학습

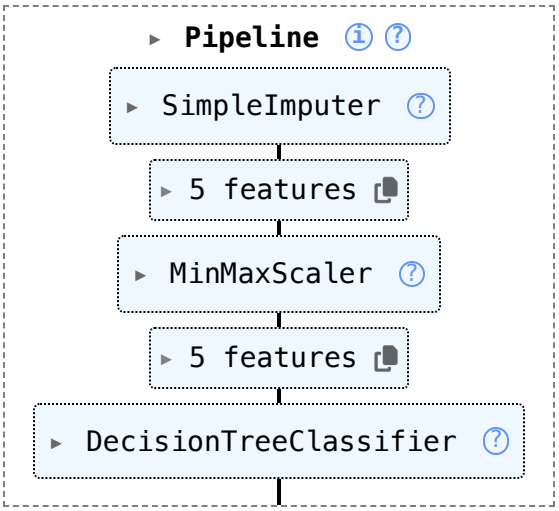
목차로 이동하기

- 각 요소의 이름은 imputer, scaler, model 로 지정.

In [8]:
```
from sklearn.pipeline import Pipeline

pipe_line = Pipeline([ ("imputer", imputer), ("scaler", scaler), ("model", m
pipe_line.fit(X_train, y_train)
```

Out[8]:



파이프라인을 이용한 예측 수행

In [9]:
```
pipe_line.predict(X_test)
```

Out[9]:
```
array([False,  True, False, ...,  True,  True,  True], shape=(2174,))
```

In [10]:
```
pipe_line.score(X_test, y_test)
```

Out[10]:
```
0.7396504139834407
```

정리

- 일부 변수를 선택 후, 결측치는 평균으로 대체된 후, 스케일링된 X_test가 DecisionTree 모델에 입력되어 나온 결과이다.
- predict 메서드는 파이프라인의 마지막 요소인 의사결정트리 모델의 메서드를 사용하였다.
- score()메서드를 활용하여 확인 결과, 74%의 정확도를 갖는다.

실습

- 파이프라인의 모델을 다른 모델(KNN)으로 변경 후, 수행해 보기.
- 모델이 앞의 모델과 비교하여 어떠한지 비교해 보기

모델의 스케일링 함수 변경해 보기

In [11]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier as KNN_C

imputer = SimpleImputer(strategy = "mean")
scaler = StandardScaler()
model = KNN_C()
```

In [12]:
```
pipe_line = Pipeline([ ("imputer", imputer),
                       ("scaler", scaler),
                       ("model", model) ])

pipe_line.fit(X_train, y_train)
pipe_line.score(X_test, y_test)
```

Out[12]:
```
0.7695492180312787
```

Pipeline 클래스의 단점

- 손쉽게 파이프라인을 구축할 수 있는 장점이 있다.
- 단, 다음과 같은 문제도 존재한다.
    - 1. 사이킷런 인스턴스가 아닌 다른 클래스의 인스턴스가 파이프라인에 포함되면 정상적으로 작동하지 않을 수 있음.
    - 2. 하이퍼파라미터 튜닝 등을 할 때 전체 파이프라인을 계속해서 수정해야 한다.
        - 해결 제안 : 함수나 클래스를 사용하여 파이프라인을 함수화 해보기

03 파이프라인 함수로 만들기

목차로 이동하기

pipe_line_fnc 함수 만들기

In [13]:
```
def pipe_line_fnc(X, imputer, scaler, model):
    X = imputer.transform(X)
    X = scaler.transform(X)
    pred = model.predict(X)
    return pred

pred_Y = pipe_line_fnc(X_test, imputer, scaler, model)

pred_Y[:5]
```

Out[13]:
```
array([False,  True, False, False,  True])
```

In [14]:
```
pred_Y.shape
```

Out[14]:
```
(2174,)
```

In [15]:
```
from sklearn.metrics import accuracy_score
accuracy_score(pred_Y, y_test)
```

Out[15]:
```
0.7695492180312787
```

최종 결과 73.59%

In [16]:
```
from sklearn.metrics import classification_report
print( classification_report(pred_Y, y_test) )
```

```
              precision    recall  f1-score   support

       False       0.69      0.81      0.75       916
        True       0.85      0.74      0.79      1258

    accuracy                           0.77      2174
   macro avg       0.77      0.78      0.77      2174
weighted avg       0.78      0.77      0.77      2174
```

- 정밀도 : 0.85, 재현율 : 0.70, f1-score : 0.76

04 학습내용 저장 및 불러오기

목차로 이동하기

In [17]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
# from sklearn.externals import joblib
import sklearn.externals
import joblib
```

In [18]:
```
sel = [ 'RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck' ]
X = train[sel]
y = train['Transported']

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)
```

In [19]:
```
imputer = SimpleImputer(strategy = "mean")
scaler = MinMaxScaler()
model = LogisticRegression()
```

In [20]:
```
pipe_line_Log = Pipeline([ ("imputer", imputer), ("scaler", scaler), ("model
pipe_line_Log.fit(X_train, y_train)
pred = pipe_line_Log.predict(X_test)
pred[0:5]
```

Out[20]:
```
array([ True,  True, False, False,  True])
```

In [21]:
```
# 정확도 확인
print( accuracy_score(pred, y_test) )
```

```
0.7378104875804968
```

In [22]:
```
model = KNeighborsClassifier()
```

In [23]:
```
pipe_line_knn = Pipeline([ ("imputer", imputer), ("scaler", scaler), ("model
pipe_line_knn.fit(X_train, y_train)
pred = pipe_line_knn.predict(X_test)

# 정확도 확인
print( accuracy_score(pred, y_test) )
```

```
0.7723091076356946
```

In [24]:
```
joblib.dump(pipe_line_knn, os.path.join(SAVE_DIR, "model_pipe_knn.joblib") )
```

Out[24]:
```
['data/Space_Titanic/model_pipe_knn.joblib']
```

In [25]:
```
import os
os.listdir(SAVE_DIR)
```

Out[25]:
```
['model_pipe_knn.joblib']
```

불러오기 후, 확인

In [26]:
```
import sklearn.externals
import joblib
```

```
pipe_knn = joblib.load(os.path.join(SAVE_DIR, "model_pipe_knn.joblib"))

pred = pipe_knn.predict(X_test)

# 정확도 확인
print( accuracy_score(pred, y_test) )
```

```
0.7723091076356946
```